CLI 实操入门

使用 Solana CLI 与 `spl-token` 理解代币全生命周期

本讲义以**命令行实操**为主线，完整覆盖：

环境检查、创建 Mint、创建 ATA、铸造、查询余额、转账、销毁、权限管理、冻结/解冻、关闭账户等核心操作。

学习笔记 / 教学讲义版

2026 年 3 月 21 日

目录

1 章节导读 **5**

2 本章会用到的核心工具 **5**
2.1 Solana CLI 是什么 5
2.2 `spl-token` 是什么 5

3 环境检查 **6**
3.1 先确认当前网络配置 6
3.1.1 命令 6
3.1.2 示例输出 6
3.1.3 你要看哪几个字段 6
3.1.4 如果不是 Devnet，怎么切换 6
3.2 确认默认钱包地址 7
3.2.1 命令 7
3.2.2 说明 7
3.3 检查 SOL 余额 7
3.3.1 命令 7
3.3.2 为什么要有 SOL 7
3.4 余额不足时请求空投 7
3.4.1 命令 7
3.4.2 空投后建议立即再查一次余额 8
3.4.3 完整环境检查最小命令集 8

4 创建代币（Mint） **8**
4.1 这一节到底在做什么 8
4.2 创建一个精度为 6 的代币 8
4.2.1 命令 8
4.2.2 示例输出 8
4.2.3 输出中最重要的是哪一项 8
4.2.4 建议的做法：把 Mint 地址记下来 9
4.3 查看刚创建的代币信息 9
4.3.1 命令 9
4.3.2 示例输出 9
4.3.3 怎么读这段输出 9

5 创建代币账户 **10**
5.1 为什么还不能直接 Mint 10
5.2 为当前钱包创建该代币的 ATA 10
5.2.1 命令 10
5.2.2 示例输出 10
5.2.3 如果账户已存在怎么办 10

5.2.4 建议把 Token Account 也记下来 11

6 铸造代币 11

6.1 这一节在做什么 11
6.2 铸造 1000 个代币到自己的账户 11
6.2.1 命令 11
6.2.2 示例输出 11
6.2.3 这一步做了什么 11
6.3 用户输入数量 vs 底层存储数量 12

7 检查余额与查询账户 12

7.1 查看某个代币的余额 12
7.1.1 命令 12
7.1.2 示例输出 12
7.1.3 为什么显示的是 1000，而不是 1000000000 12
7.2 查看当前钱包下的所有代币账户 13
7.2.1 命令 13
7.2.2 示例输出 13
7.2.3 这条命令的价值 13
7.3 再次查看 Mint 信息确认供应量 13

8 转账 13

8.1 这一节在做什么 13
8.2 转账 100 个代币给其他地址 14
8.2.1 命令 14
8.2.2 示例输出 14
8.3 这里传的是钱包地址，不一定是 Token Account 地址 14
8.4 `-fund-recipient` 的作用 14
8.5 转账后建议做的检查 14

9 销毁代币（Burn） 15

9.1 Burn 的本质 15
9.2 销毁 50 个代币 15
9.2.1 方式一：使用 Token Account 地址 15
9.2.2 方式二：直接使用 Mint 地址 15
9.3 为什么两种写法都能用 15
9.4 销毁后确认供应量变化 15
9.5 销毁后还建议再查什么 16

10 权限管理 16

10.1 为什么权限管理很重要 16
10.2 查看当前权限 16
10.3 移除铸造权限 16
10.3.1 命令 16

10.3.2 这条命令的效果 . . . 17
10.4 把铸造权限转移给其他地址 . . . 17
10.5 移除冻结权限 . . . 17
10.5.1 为什么有人会移除冻结权限 . . . 17
10.5.2 权限管理命令汇总 . . . 17

11 冻结与解冻账户 17
11.1 冻结功能的前提 . . . 17
11.2 冻结账户 . . . 18
11.2.1 命令 . . . 18
11.2.2 效果 . . . 18
11.3 解冻账户 . . . 18
11.3.1 命令 . . . 18
11.4 冻结/解冻的常见场景 . . . 18

12 关闭账户 18
12.1 为什么要关闭 Token Account . . . 18
12.2 关闭前的前提条件 . . . 18
12.3 先检查余额 . . . 19
12.4 关闭账户并把租金退回当前钱包 . . . 19
12.5 关闭账户并指定接收地址 . . . 19
12.6 关闭账户后会发生什么 . . . 19

13 推荐的完整实操顺序 19
13.1 第一阶段：环境确认 . . . 19
13.2 第二阶段：创建 Mint . . . 19
13.3 第三阶段：创建接收账户 . . . 20
13.4 第四阶段：铸造并检查 . . . 20
13.5 第五阶段：转账 . . . 20
13.6 第六阶段：销毁 . . . 20
13.7 第七阶段：权限与高级操作 . . . 20
13.8 第八阶段：关闭闲置账户 . . . 20

14 本章所有必备终端命令总表 20
14.1 Solana CLI 命令 . . . 20
14.2 `spl-token`：Mint 与查询 . . . 21
14.3 `spl-token`：账户与铸造 . . . 21
14.4 `spl-token`：转账 . . . 21
14.5 `spl-token`：销毁 . . . 21
14.6 `spl-token`：权限 . . . 21
14.7 `spl-token`：冻结 / 解冻 / 关闭 . . . 21

15 常见问题与易错点 21
15.1 为什么创建了 Mint 之后 Supply 还是 0? . . . 21

15.2 为什么我不能直接往钱包地址里存 SPL Token? 21
15.3 为什么转账时要考虑接收方 ATA? 22
15.4 为什么关闭账户前余额必须为 0? 22
15.5 为什么移除 Mint Authority 是大事? 22

16 本章总结 **22**

附录：一页速查版 **22**

1 章节导读

本章目标不是“背命令”，而是通过命令行亲手操作，真正理解以下问题：

1. Solana CLI 当前连的是哪个网络？
2. 钱包地址、Mint、Token Account、ATA 在实际命令里分别对应什么？
3. 创建一个新代币时，链上到底发生了哪些事情？
4. 铸造、转账、销毁、冻结、关闭账户这些动作分别改动了哪些状态？

在 Solana 开发中，CLI 是理解底层结构最快的入口。因为 UI 钱包会帮你隐藏很多细节，而命令行会直接把“创建了什么账户、调用了什么程序、消耗了什么权限”展示出来。

本章默认你已经完成：

- Solana CLI 安装
- `spl-token` 工具安装
- Devnet 配置
- 本地钱包创建
- 钱包里有少量测试 SOL

2 本章会用到的核心工具

2.1 Solana CLI 是什么

`solana` 是与 Solana 网络交互的基础命令行工具。你会用它来：

- 查看当前网络配置；
- 查看默认钱包地址；
- 查询 SOL 余额；
- 请求 Devnet 空投；
- 确认交易状态。

2.2 `spl-token` 是什么

`spl-token` 是专门用来操作 SPL Token 的命令行工具。你会用它来：

- 创建新的代币 Mint；
- 创建 Token Account / ATA；
- 铸造代币；
- 查询代币余额；
- 转账代币；
- 销毁代币；

- 修改权限；
- 冻结、解冻、关闭账户。

可以这样记：
- solana：更偏“链与钱包层”
- spl-token：更偏“代币资产层”

3 环境检查

3.1 先确认当前网络配置

第一步一定是确认 CLI 当前到底连到哪里。因为如果你误连到 Mainnet、Localnet 或 Testnet，后面所有操作的结果都会不一样。

3.1.1 命令

```
# 检查当前配置
solana config get
```

3.1.2 示例输出

```
# Config File: ~/.config/solana/cli/config.yml
# RPC URL: https://api.devnet.solana.com
# WebSocket URL: wss://api.devnet.solana.com/
# Keypair Path: ~/.config/solana/id.json
# Commitment: confirmed
```

3.1.3 你要看哪几个字段

字段	含义
RPC URL	当前连接的网络。做本章实操时应为 devnet
Keypair Path	默认钱包私钥文件路径
Commitment	查询确认级别，常见为 confirmed

3.1.4 如果不是 Devnet，怎么切换

```
solana config set --url devnet
solana config get
```

做练习时一定尽量使用 **Devnet**。因为本章很多操作会消耗测试 SOL，使用 Mainnet 会产生真实成本。

3.2 确认默认钱包地址

接下来要确认：当前 CLI 使用的是哪个地址作为默认签名者。

3.2.1 命令

```
# 确认钱包地址
solana address
```

3.2.2 说明

这个地址就是：

- 你后续创建 Mint 时的默认付款方；
- 你后续铸造代币时可能的默认接收方；
- 你后续修改权限时的默认签名地址。

如果这里报错 `No default signer found`，说明你还没有正确创建或配置默认钱包。

3.3 检查 SOL 余额

有了默认钱包之后，还要确认里面有足够的 SOL 来支付交易费和账户租金。

3.3.1 命令

```
# 检查余额（需要 SOL 支付交易费用）
solana balance
```

3.3.2 为什么要有 SOL

即使你操作的是 SPL Token，链上的每一步仍然可能涉及：

- 发起交易
- 创建账户
- 支付 rent-exempt 最低余额

这些都需要 SOL。

3.4 余额不足时请求空投

如果当前网络是 Devnet，就可以请求测试 SOL。

3.4.1 命令

```
# 如果余额不足，申请空投
solana airdrop 2
```

3.4.2 空投后建议立即再查一次余额

```
solana balance
```

3.4.3 完整环境检查最小命令集

```
solana config get
solana address
solana balance
solana airdrop 2
solana balance
```

到这一步，你应该已经确认：

- 当前在 Devnet
- 默认钱包存在
- 钱包地址可正常读取
- 账户里有足够测试 SOL

4 创建代币（Mint）

4.1 这一节到底在做什么

“创建代币”在 Solana 上并不是部署一份新的 ERC-20 风格合约。你真正做的是：

1. 调用 Token Program；
2. 创建一个新的 Mint Account；
3. 在这个 Mint Account 中写入该代币的配置。

4.2 创建一个精度为 6 的代币

4.2.1 命令

```
# 创建代币，精度为 6 位小数
spl-token create-token --decimals 6
```

4.2.2 示例输出

```
# Creating token 7xKXtg2CW87d97TXJSDpbD5jBkheTqA83TZRuJosgAsU
# Address:  7xKXtg2CW87d97TXJSDpbD5jBkheTqA83TZRuJosgAsU
# Decimals:  6
# Signature: 5Uv2M...
```

4.2.3 输出中最重要的是哪一项

最重要的是：

```
Address:  7xKXtg2CW87d97TXJSDpbD5jBkheTqA83TZRuJosgAsU
```

这就是你新创建的 **Mint 地址**。

后续你几乎所有命令都会围绕它展开，所以建议你把它存成变量或记在笔记里。

4.2.4 建议的做法：把 Mint 地址记下来

```
# 可选：把 Mint 地址记成环境变量，后面更方便
export MINT=<MINT_ADDRESS>
echo $MINT
```

例如：

```
export MINT=7xKXtg2CW87d97TXJSDpbD5jBkheTqA83TZRuJosgAsU
echo $MINT
```

4.3 查看刚创建的代币信息

创建之后，不要急着往下做，先看清楚 Mint 里到底写了什么。

4.3.1 命令

```
spl-token display <MINT_ADDRESS>
```

如果你用了变量：

```
spl-token display $MINT
```

4.3.2 示例输出

```
# SPL Token Mint
#   Address: 7xKXtg2CW87d97TXJSDpbD5jBkheTqA83TZRuJosgAsU
#   Program: TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA
#   Supply: 0
#   Decimals: 6
#   Mint authority: <YOUR_WALLET_ADDRESS>
#   Freeze authority: <YOUR_WALLET_ADDRESS>
```

4.3.3 怎么读这段输出

字段	解释
`Address`	这个 Mint Account 自己的地址
`Program`	管理这个代币的 Token Program 地址
`Supply`	当前总供应量，此时是 0，因为你还没有铸造
`Decimals`	该代币的小数位数，这里是 6

`Mint authority`	谁有权继续铸造新代币
`Freeze authority`	谁有权冻结相关 Token Account

此时 `Supply = 0` 非常正常。因为你只是定义了“这种币”，还没有真正把它发到任何账户里。

5 创建代币账户

5.1 为什么还不能直接 Mint

很多初学者会问：

> “我不是已经创建代币了吗？为什么还要再创建账户？”

原因是：Mint Account 只定义“币种”，并不负责存余额。所以在你把代币发出去之前，必须先有一个地方来接收它，也就是：

Token Account / ATA

5.2 为当前钱包创建该代币的 ATA

5.2.1 命令

```
# 为当前钱包创建该代币的 ATA
spl-token create-account <MINT_ADDRESS>
```

如果你用了变量：

```
spl-token create-account $MINT
```

5.2.2 示例输出

```
# Creating account 9WzDXwBbmkg8ZTbNMqUxvQRAyrZzDsGYdLVL9zYtAWWM
# Signature: 3fE...
```

这个 `Creating account ...` 后面的地址，就是你当前钱包针对这个 Mint 的一个代币账户地址，通常就是默认 ATA。

5.2.3 如果账户已存在怎么办

如果该账户已经创建过，再执行命令通常会提示账户已存在。这不一定是错误，只说明：

你针对该 Mint 的默认接收账户已经有了。

5.2.4 建议把 Token Account 也记下来

```
# 可选：把刚创建的 token account 也记录下来
export TOKEN_ACCOUNT=<TOKEN_ACCOUNT_ADDRESS>
echo $TOKEN_ACCOUNT
```

严格来说，`spl-token create-account` 创建的是“该钱包针对某个 Mint 的代币账户”。在绝大多数标准场景里，你可以把它理解成你的默认 ATA。

6 铸造代币

6.1 这一节在做什么

“铸造”就是把新的代币单位真正创建出来，并发到某个接收账户里。

注意这里的逻辑顺序：

1. 你先定义了一种币（Mint）
2. 你再准备了一个接收该币的账户（Token Account / ATA）
3. 然后你才可以执行 Mint

6.2 铸造 1000 个代币到自己的账户

6.2.1 命令

```
# 铸造 1000 个代币到自己的账户
spl-token mint <MINT_ADDRESS> 1000
```

如果你用了变量：

```
spl-token mint $MINT 1000
```

6.2.2 示例输出

```
# Minting 1000 tokens
#   Token: 7xKXtg2CW87d97TXJSDpbD5jBkheTqA83TZRuJosgAsU
#   Recipient: 9WzDXwBbmkg8ZTbNMqUxvQRAyrZzDsGYdLVL9zYtAWWM
# Signature: 4gH...
```

6.2.3 这一步做了什么

这条命令做的不是“改 UI 数字”，而是：

- 增加 Mint 的 `Supply`
- 把对应数量写入接收方的 Token Account

也就是说，链上至少发生了两个状态变化：

1. 总供应量增加
2. 某个 Token Account 的余额增加

6.3 用户输入数量 vs 底层存储数量

这里输入的 `1000` 是**用户可读数量**。如果你创建 Mint 时设置的是：

$$decimals = 6$$

那么底层实际增加的整数数量是：

$$1000 \times 10^6 = 1{,}000{,}000{,}000$$

也就是说：
- 你在终端输入：`1000`
- 你在钱包里看到：`1000.000000`
- 链上底层实际记录：`1000000000`

7 检查余额与查询账户

7.1 查看某个代币的余额

铸造完成后，先检查这个 Mint 对应的余额是否正确。

7.1.1 命令

```
# 查看特定代币余额
spl-token balance <MINT_ADDRESS>
```

或者：

```
spl-token balance $MINT
```

7.1.2 示例输出

```
# 输出：1000
```

7.1.3 为什么显示的是 1000，而不是 1000000000

因为 CLI 会根据 Mint 的 `decimals` 自动帮你做人类可读格式化。所以你看到的是“用户视角余额”，而不是链上最小单位整数。

7.2 查看当前钱包下的所有代币账户

7.2.1 命令

```
# 查看所有代币余额
spl-token accounts
```

7.2.2 示例输出

```
# Token                                            Balance
# ------------------------------------------------------------
# 7xKXtg2CW87d97TXJSDpbD5jBkheTqA83TZRuJosgAsU  1000
```

7.2.3 这条命令的价值

这条命令有助于你快速知道：

- 当前钱包手上有哪些 SPL 代币；
- 各种代币当前余额是多少；
- 某个 Mint 是否已经成功进入你的账户体系。

7.3 再次查看 Mint 信息确认供应量

铸造后，建议再查一次 Mint。

```
spl-token display <MINT_ADDRESS>
```

或：

```
spl-token display $MINT
```

你会看到 `Supply` 不再是 0，而会变成对应的新总供应量。

> 查看余额是从“用户账户视角”看状态。查看 `display` 则是从“Mint 全局配置视角”看状态。两者一起看，最容易理解代币系统。

8 转账

8.1 这一节在做什么

转账就是把某个 Token Account 里的余额，转移到另一个用户对应该 Mint 的 Token Account 中。

与 SOL 转账不同，SPL Token 转账涉及的是：

- 代币类型（Mint）
- 发送方该 Mint 的 Token Account

- 接收方该 Mint 的 Token Account / ATA

8.2 转账 100 个代币给其他地址

8.2.1 命令

```
# 转账 100 个代币给目标钱包
# 如果对方没有该代币的 ATA，可以添加 --fund-recipient 自动创建
spl-token transfer <MINT_ADDRESS> 100 <RECIPIENT_WALLET> --fund-recipient
```

8.2.2 示例输出

```
# Transfer 100 tokens
#   Sender: 9WzDXwBbmkg8ZTbNMqUxvQRAyrZzDsGYdLVL9zYtAWWM
#   Recipient: Fg6PaFpoGXkYsidMpWxqSk3V7HFgyNvXz9hLYC7e123
# Signature: 2sY...
```

8.3 这里传的是钱包地址，不一定是 Token Account 地址

命令里你通常传入的是：

`<RECIPIENT_WALLET>`

而不是接收方的 Token Account 地址本身。

原因是 `spl-token` 可以结合 Mint 信息，自动帮你找到或创建接收方该代币对应的标准账户。

8.4 `-fund-recipient` 的作用

`-fund-recipient` 非常重要，它的意思是：

> 如果接收方还没有这个 Mint 对应的 ATA，那就由发送方帮他创建。

这时发送方会额外支付：

- 创建接收方 ATA 的账户租金
- 相关交易费

如果不加 `-fund-recipient`，而对方又没有对应的 ATA，转账可能失败。

8.5 转账后建议做的检查

转账后建议至少检查两件事：

```
# 查看自己余额是否减少
spl-token balance <MINT_ADDRESS>

# 查看全部资产列表
spl-token accounts
```

如果你能让对方也执行：

```
spl-token accounts
```

那么就更容易验证转账是否确实到账。

9 销毁代币（Burn）

9.1 Burn 的本质

销毁不是“隐藏余额”，也不是“锁仓”。销毁的意思是：

永久减少某个 Token Account 中的数量，并同步减少总供应量。

9.2 销毁 50 个代币

9.2.1 方式一：使用 Token Account 地址

```
# 销毁 50 个代币
spl-token burn <TOKEN_ACCOUNT_ADDRESS> 50
```

9.2.2 方式二：直接使用 Mint 地址

```
# 或者使用 Mint 地址（会自动找到 ATA）
spl-token burn <MINT_ADDRESS> 50
```

如果你用了变量：

```
spl-token burn $MINT 50
```

9.3 为什么两种写法都能用

因为 `spl-token` 在标准场景下能根据：

- 当前默认钱包
- Mint 地址

自动推断出你对应的默认 Token Account / ATA。

9.4 销毁后确认供应量变化

```
spl-token supply <MINT_ADDRESS>
```

或者：

```
spl-token supply $MINT
```

9.5 销毁后还建议再查什么

```
# 查自己的余额
spl-token balance <MINT_ADDRESS>

# 查 Mint 全局信息
spl-token display <MINT_ADDRESS>
```

Burn 会同时影响两层状态：

- 你的 Token Account 余额减少
- Mint 的总供应量减少

10 权限管理

10.1 为什么权限管理很重要

Solana 上很多代币风险，不是出在 Token Program 本身，而是出在权限配置上。例如：

- 如果 `mint authority` 还在，项目方理论上就还能继续增发；
- 如果 `freeze authority` 还在，项目方理论上还能冻结相关账户。

所以学会查看和修改权限，是理解代币可信度的核心步骤。

10.2 查看当前权限

```
# 查看代币权限
spl-token display <MINT_ADDRESS>
```

或：

```
spl-token display $MINT
```

在输出里重点观察：

- `Mint authority`
- `Freeze authority`

10.3 移除铸造权限

10.3.1 命令

```
# 移除铸造权限（不可逆！）
spl-token authorize <MINT_ADDRESS> mint --disable
```

或：

```
spl-token authorize $MINT mint --disable
```

10.3.2 这条命令的效果

一旦执行成功，`mint authority` 会被设为无效状态。之后这个 Mint 将不能再继续增发新币。

这通常是不可逆操作。所以在真实项目里，只有在你非常确定总量应固定时才会这么做。

10.4 把铸造权限转移给其他地址

```
# 将铸造权限转移给其他地址
spl-token authorize <MINT_ADDRESS> mint <NEW_AUTHORITY>
```

例如：

```
spl-token authorize $MINT mint <NEW_AUTHORITY>
```

这在以下场景里常见：

- 项目方把权限交给治理金库
- 项目方把权限转给多签
- 开发测试时切换到另一个控制地址

10.5 移除冻结权限

```
# 移除冻结权限
spl-token authorize <MINT_ADDRESS> freeze --disable
```

或：

```
spl-token authorize $MINT freeze --disable
```

10.5.1 为什么有人会移除冻结权限

因为一旦保留冻结权限，代币发行方理论上还能冻结某些账户。对于强调去中心化或不可审查性的项目，常会选择把这一权限移除。

10.5.2 权限管理命令汇总

```
spl-token display <MINT_ADDRESS>
spl-token authorize <MINT_ADDRESS> mint --disable
spl-token authorize <MINT_ADDRESS> mint <NEW_AUTHORITY>
spl-token authorize <MINT_ADDRESS> freeze --disable
```

11 冻结与解冻账户

11.1 冻结功能的前提

只有当该 Mint 仍然保留了 `freeze authority` 时，冻结相关 Token Account 才有可能成功。

如果冻结权限已经被移除，那么后续就不能再使用冻结/解冻功能。

11.2 冻结账户

11.2.1 命令

```
# 冻结账户（被冻结的账户无法转出代币）
spl-token freeze <TOKEN_ACCOUNT_ADDRESS>
```

11.2.2 效果

被冻结的 Token Account 里的代币通常不能正常转出。这是针对“账户状态”的控制，而不是把 Mint 整体停掉。

11.3 解冻账户

11.3.1 命令

```
# 解冻账户
spl-token thaw <TOKEN_ACCOUNT_ADDRESS>
```

11.4 冻结/解冻的常见场景

- 合规型稳定币风控
- 被盗资产的紧急处理
- 法务要求下的限制性措施

例如在合规稳定币场景中，如果某地址涉嫌非法活动，发行方可能会冻结其 Token Account，使这些代币暂时不能自由转移。

12 关闭账户

12.1 为什么要关闭 Token Account

在 Solana 上，账户不是完全免费的。创建 Token Account 时，通常需要存入一部分 SOL 作为租金豁免余额（rent-exempt reserve）。

如果某个 Token Account 之后已经不用了，而且余额已经清空，就可以把它关闭，以回收这部分 SOL。

12.2 关闭前的前提条件

关闭账户之前，必须确保：

该 Token Account 的余额为 0

否则关闭会失败。

12.3 先检查余额

```
# 首先确保账户余额为 0
spl-token balance <MINT_ADDRESS>
```

如果余额不为 0，需要先：

- 转走代币
- 或者销毁代币

12.4 关闭账户并把租金退回当前钱包

```
# 关闭账户，租金退还到当前钱包
spl-token close <TOKEN_ACCOUNT_ADDRESS>
```

12.5 关闭账户并指定接收地址

```
# 或关闭并指定接收地址
spl-token close <TOKEN_ACCOUNT_ADDRESS> --recipient <WALLET_ADDRESS>
```

12.6 关闭账户后会发生什么

- 账户状态被回收；
- 租金豁免余额返还给指定地址；
- 该 Token Account 不再存在。

关闭账户回收的是 **SOL 租金**，不是“额外奖励”。所以不要把关闭账户理解成赚币，而是把闲置账户占用的 SOL 取回来。

13 推荐的完整实操顺序

如果你要完整做一次“从 0 到 1 的发币实验”，推荐按下面顺序执行。

13.1 第一阶段：环境确认

```
solana config get
solana address
solana balance
solana airdrop 2
solana balance
```

13.2 第二阶段：创建 Mint

```
spl-token create-token --decimals 6
spl-token display <MINT_ADDRESS>
```

13.3 第三阶段：创建接收账户

```
spl-token create-account <MINT_ADDRESS>
```

13.4 第四阶段：铸造并检查

```
spl-token mint <MINT_ADDRESS> 1000
spl-token balance <MINT_ADDRESS>
spl-token accounts
spl-token display <MINT_ADDRESS>
```

13.5 第五阶段：转账

```
spl-token transfer <MINT_ADDRESS> 100 <RECIPIENT_WALLET> --fund-recipient
spl-token balance <MINT_ADDRESS>
```

13.6 第六阶段：销毁

```
spl-token burn <MINT_ADDRESS> 50
spl-token balance <MINT_ADDRESS>
spl-token supply <MINT_ADDRESS>
```

13.7 第七阶段：权限与高级操作

```
spl-token display <MINT_ADDRESS>
spl-token authorize <MINT_ADDRESS> mint --disable
spl-token authorize <MINT_ADDRESS> freeze --disable
```

13.8 第八阶段：关闭闲置账户

```
spl-token close <TOKEN_ACCOUNT_ADDRESS>
spl-token close <TOKEN_ACCOUNT_ADDRESS> --recipient <WALLET_ADDRESS>
```

14 本章所有必备终端命令总表

下面把本章涉及的所有核心命令集中列出，便于你直接抄到笔记里。

14.1 Solana CLI 命令

```
solana config get
solana config set --url devnet
solana address
solana balance
solana airdrop 2
```

14.2 spl-token：Mint 与查询

```
spl-token create-token --decimals 6
spl-token display <MINT_ADDRESS>
spl-token supply <MINT_ADDRESS>
spl-token accounts
spl-token balance <MINT_ADDRESS>
```

14.3 spl-token：账户与铸造

```
spl-token create-account <MINT_ADDRESS>
spl-token mint <MINT_ADDRESS> 1000
```

14.4 spl-token：转账

```
spl-token transfer <MINT_ADDRESS> 100 <RECIPIENT_WALLET> --fund-recipient
```

14.5 spl-token：销毁

```
spl-token burn <TOKEN_ACCOUNT_ADDRESS> 50
spl-token burn <MINT_ADDRESS> 50
```

14.6 spl-token：权限

```
spl-token authorize <MINT_ADDRESS> mint --disable
spl-token authorize <MINT_ADDRESS> mint <NEW_AUTHORITY>
spl-token authorize <MINT_ADDRESS> freeze --disable
```

14.7 spl-token：冻结 / 解冻 / 关闭

```
spl-token freeze <TOKEN_ACCOUNT_ADDRESS>
spl-token thaw <TOKEN_ACCOUNT_ADDRESS>
spl-token close <TOKEN_ACCOUNT_ADDRESS>
spl-token close <TOKEN_ACCOUNT_ADDRESS> --recipient <WALLET_ADDRESS>
```

15 常见问题与易错点

15.1 为什么创建了 Mint 之后 Supply 还是 0？

因为创建 Mint 只是“定义这种币”，并没有把币真正铸造出来。只有执行 `spl-token mint` 后，总供应量才会增加。

15.2 为什么我不能直接往钱包地址里存 SPL Token？

因为钱包地址主要是 System Account。SPL Token 余额实际存放在 Token Account / ATA 中。

15.3 为什么转账时要考虑接收方 ATA？

因为对方如果没有该 Mint 对应的 Token Account，系统就没有地方记这笔余额。`-fund-recipient` 就是为了解决这个问题。

15.4 为什么关闭账户前余额必须为 0？

因为关闭账户代表“回收这个容器”。如果容器里还装着代币，就不能直接回收。

15.5 为什么移除 Mint Authority 是大事？

因为一旦移除，未来就再也不能增发。这通常意味着该代币将转为固定总量模型。

16 本章总结

如果把本章浓缩成一句话：

Solana CLI 让你看到链与钱包状态，`spl-token` 让你亲手操作代币生命周期。

请牢牢记住下面这个流程：

1. 先确认网络与钱包；
2. 再创建 Mint；
3. 再创建 Token Account / ATA；
4. 再铸造；
5. 再查询余额；
6. 再做转账、销毁、权限管理与账户清理。

你真正学到的不只是命令本身，而是命令背后的结构：

- **Mint** 定义币种；
- **Token Account / ATA** 存余额；
- **MintTo** 增加供应量；
- **Transfer** 在账户之间移动余额；
- **Burn** 减少余额与总供应量；
- **Authorize / Freeze / Close** 则体现权限与账户生命周期管理。

附录：一页速查版

最小必备命令序列

```
# 1. 环境检查
solana config get
solana address
solana balance
```

```
solana airdrop 2

# 2. 创建 Mint
spl-token create-token --decimals 6
spl-token display <MINT_ADDRESS>

# 3. 创建代币账户
spl-token create-account <MINT_ADDRESS>

# 4. 铸造
spl-token mint <MINT_ADDRESS> 1000
spl-token balance <MINT_ADDRESS>
spl-token accounts

# 5. 转账
spl-token transfer <MINT_ADDRESS> 100 <RECIPIENT_WALLET> --fund-recipient

# 6. 销毁
spl-token burn <MINT_ADDRESS> 50
spl-token supply <MINT_ADDRESS>

# 7. 权限管理
spl-token display <MINT_ADDRESS>
spl-token authorize <MINT_ADDRESS> mint --disable
spl-token authorize <MINT_ADDRESS> freeze --disable

# 8. 关闭闲置账户
spl-token close <TOKEN_ACCOUNT_ADDRESS>
```